

# Assignment #3: AI-Assisted Fiscal Policy Dashboard

## ECO 317 - Intermediate Macroeconomic Theory

Professor Jonathan Wolff

Spring 2026

**Format:** Group Project (Max 6 students per group)

**Tools:** Python, Streamlit, Generative AI (ChatGPT/Gemini/Claude)

### 1. Overview

In our first assignment, we focused on confronting theory with real-world evidence by building an automated data pipeline. In our second assignment, we shifted to simulating theoretical mechanics by solving dynamic programming problems over an infinite horizon.

Now, we bring these concepts together into a comprehensive general equilibrium framework to evaluate policy. Evaluating the effectiveness of government intervention—specifically fiscal policy—requires more than just a theoretical hunch; it requires a structural model capable of capturing the complex, interconnected frictions of the real economy. Furthermore, before a model can be trusted to evaluate a policy shock, it must first prove it can replicate the unconditional, stylized facts of the business cycle.

In this assignment, you will use Generative AI tools to construct an interactive, web-based DSGE (Dynamic Stochastic General Equilibrium) Dashboard using Python and Streamlit. Your application will simulate a medium-scale model inspired by the landmark Smets and Wouters (2007) framework to analyze the macroeconomic effects of fiscal stimulus, tax cuts, and debt reduction.

The catch: You are not expected to write every line of Python from scratch. You are expected to act as economic architects. Use AI to do the heavy lifting of solving the linear rational expectations systems (e.g., using QZ decomposition or established Python DSGE libraries) while you ensure the economic logic, the budget constraints, and the multiplier mathematics remain perfectly sound.

### 2. Project Requirements

Your Streamlit application must successfully feature a dynamic, customizable medium-scale DSGE model integrated into a unified interactive dashboard. The dashboard must utilize a **tabbed interface** to separate the unconditional model evaluation from the conditional policy experiments.

## Phase I: The Global Sidebar & Structural Engine

The background Python code must simulate a linearized Smets and Wouters (2007) style economy. The global sidebar of your app must contain continuous sliders (scroll bars) to control the core structural parameters of the economy:

- **Meaningful Debt & Frictions:** The model must include a fraction of "Rule-of-Thumb" (non-Ricardian) households that simply consume their current after-tax income, alongside standard forward-looking optimizing households.
- **Continuous Frictions:** You must include sliders to dictate the intensity of the following features:
  - Habit formation in consumption ( $h$ ).
  - Variable capital utilization (cost convexity parameter  $\psi$ ).
  - Price stickiness (Calvo parameter  $\theta_p$ ).
  - Wage stickiness (Calvo parameter  $\theta_w$ ).
  - Debt-feedback aggressiveness ( $\phi_b$ ).

## Phase II: Tab 1 - Model Fit (Unconditional Dynamics)

Before analyzing policy, you must prove the engine works. This tab will simulate the economy's response to random structural shocks over a long horizon (e.g., 1,000 periods).

- **Key Simulated Moments:** The app must dynamically calculate and display at least 10 key business cycle moments, including the volatilities (variances) of Output, Consumption, Investment, Labor, and Inflation; their contemporaneous correlations with Output; and first-order autocorrelations.
- **AI Output Interpretation:** Incorporate a dynamic text block utilizing Python f-strings to automatically generate written economic analysis comparing the simulated moments to stylized macroeconomic facts (e.g., explaining how the habit formation slider successfully forces Consumption volatility to be lower than Output volatility).

## Phase III: Tab 2 - Fiscal Exercises (Conditional Policy)

Once the model's baseline fit is established, this tab will simulate the economy's response to specific fiscal interventions over a 40-quarter horizon.

- **The Shocks:** The user must be able to trigger specific fiscal shocks: an increase in government consumption spending, an increase in government investment spending, and isolated cuts to labor or capital taxes.
- **Variable Financing Rules:** The user must be able to select *how* the government stabilizes the resulting debt via a radio button or dropdown menu. Options must include: Lump-Sum

(Ricardian) transfers, Consumption Tax Hikes, Labor Tax Hikes, Capital Tax Hikes, or Government Spending Cuts.

- **Multiplier Calculations:** The app must dynamically calculate and display both the **Impact Multiplier** and the **Cumulative Multiplier**.
- **Visuals:** Display clean, well-labeled visualizations of the Impulse Response Functions (IRFs) for Output, Consumption, Investment, and Government Debt. Include multiple subplots to easily compare how variables respond simultaneously.
- **Automated Policy Briefing:** Incorporate a dynamic text block that synthesizes the multiplier analysis, explains the transmission mechanism of the selected financing rule, and calculates the "Fiscal Drag Horizon" (the exact quarter when distortionary financing forces Output below the steady state).

#### Phase IV: The Streamlit Dashboard (Visualization)

- **Academic Styling:** The dashboard must be visually polished, featuring a dark/navy background, bright white charting boxes (or white text on dark charts) for contrast, and utilizing the Garamond font throughout. Ensure all UI text, including tabs and widget labels, remains highly readable.

### 3. AI Prompting Guide & Helpful Tips

Solving medium-scale DSGE models in Python requires specific mathematical techniques. You must be highly specific when prompting ChatGPT, Gemini, or Claude.

- **Key Phrases to Include in Prompts:**
  - "Express the linearized DSGE equations in state-space form and solve the linear rational expectations model using Blanchard-Kahn conditions or the generalized Schur (QZ) decomposition (e.g., using `scipy.linalg.ordqz`)."
  - "Create a dynamic function that recalculates the steady state and the transition matrices every time a Streamlit slider parameter is changed. Ensure you use `@st.cache_data` so the app does not freeze."
  - "Simulate a 1,000-period business cycle using random normal shocks to calculate unconditional variances and cross-correlations."
  - "Calculate the cumulative fiscal multiplier as the discounted sum of the output response divided by the discounted sum of the government spending shock."

#### 4. The "Open Source" & AI Approach

- **AI Usage:** You are explicitly encouraged to use AI (ChatGPT, Gemini, Claude) to generate your Python and Streamlit code. However, you must understand how it works. If the AI hallucinates a mathematical function, fails to enforce the government budget constraint, or misspecifies the Calvo pricing equation, you must catch it and fix it (using AI, not your own programming).
- **Documentation:** Your code should be heavily commented. You can use AI to generate these comments to explain what each mathematical block does.

#### 5. Deliverable: The Live Demo

There is no written report for this assignment. Instead, your grade is based entirely on a live demonstration.

- **Scheduling:** Teams will sign up for a 10-minute appointment slot with me.
- **The Demo:** You will bring your laptop, open your terminal, run `streamlit run app.py` from scratch in front of me, and show that it works.
  - Does the dashboard load cleanly?
  - Do the interactive sliders update the moments, IRFs, and Multiplier tables in real-time?
  - Are the simulated impulse responses mathematically accurate to the macroeconomic theories we discussed in class?
- **Q&A:** I will ask random group members specific questions about the economic logic (e.g., "Explain how your prompt incorporated rule-of-thumb consumers into the aggregate consumption Euler equation," or "Why does shifting the financing rule from Lump-Sum to Capital Tax Hikes crash the cumulative multiplier?") and the workflow.